
Relatório de Auditoria Técnica e Roadmap de Evolução: Dungeon of Shadows

Este documento detalha uma análise profunda sobre o estado atual do motor de jogo Dungeon of Shadows, identificando gargalos técnicos e propondo 10 soluções de engenharia para elevar o nível do projeto.

Análise Detalhada de Melhorias

1. Implementação de Delta Time (dt) para Consistência de Movimento

O loop principal utiliza `requestAnimationFrame`, que tenta rodar a 60 FPS, mas varia conforme o hardware. Em um monitor de 144Hz, a velocidade do jogador e a gravidade seriam processadas muito mais rápido, quebrando a dificuldade do jogo. A melhoria consiste em calcular o tempo decorrido entre cada frame e usar esse fator para normalizar todas as atualizações de posição.

2. Otimização do Pipeline de Iluminação Dinâmica

O jogo utiliza uma técnica de destination-out para criar o efeito de luz em tempo real. Realizar cálculos de gradiente radial para cada tocha de parede e para o jogador em cada frame consome ciclos de processamento desnecessários. A solução recomendada é o uso de um Canvas auxiliar (offscreen) para cachear as luzes estáticas, reduzindo drasticamente o esforço da GPU.

3. Refinamento da Detecção de Colisão AABB (Eixos Separados)

A função de colisão atual apenas verifica se dois retângulos se sobrepõem de forma binária. Isso pode causar comportamentos onde o jogador fica preso em quinas ou atravessa paredes se estiver em alta velocidade. A melhoria propõe separar a resolução de colisão nos eixos X e Y de forma independente, garantindo deslizes suaves em superfícies verticais.

4. Implementação de Sistema de Knockback (Recoio por Dano)

Atualmente, o dano reinicia a posição do jogador de forma abrupta. Para melhorar a jogabilidade, deve-se implementar um vetor de força contrária no momento da colisão com inimigos. Isso empurra o jogador para trás, criando uma penalidade física que serve como feedback imediato de erro, sem necessariamente interromper o fluxo do jogo.

5. Arquitetura de Máquina de Estados Finita (FSM)

O gerenciamento de telas como Menu, Jogando, Morte e Vitória está espalhado em condicionais simples. Estruturar o jogo como uma FSM organiza o código de forma que cada estado tenha seus próprios métodos de atualização e desenho, facilitando a inclusão de novas telas e transições animadas entre fases.

6. Adição de Camadas de Foreground Parallax

O jogo já possui um sistema de parallax para o fundo. A inclusão de uma camada de primeiro plano (elementos que passam na frente do jogador, como grades ou fumaça) criaria uma sensação de profundidade tridimensional muito mais rica, utilizando velocidades de deslocamento calculadas em relação à posição da câmera.

7. Efeito de Screen Shake (Tremor de Câmera)

A percepção de impacto é essencial em jogos de plataforma. Adicionar um deslocamento aleatório temporário nas coordenadas de visualização da câmera ao sofrer dano ou realizar saltos altos daria um peso maior às interações, tornando a experiência visual muito mais dinâmica e menos estática.

8. Parametrização via Objeto de Configuração Global

Valores como gravidade, força do pulo e velocidade de movimento estão espalhados pelo código. Mover esses parâmetros para um objeto de configuração centralizado (ou um arquivo JSON externo) permite que o balanceamento do jogo seja ajustado em segundos, seguindo as melhores práticas de Clean Code e desacoplamento.

9. Sistema de Partículas de Ambiente e Ação

O sistema de faíscas atual é focado apenas nas tochas. Expandir essa lógica para gerar pequenas partículas de poeira quando o

jogador aterrissa, ou fragmentos visuais quando um inimigo desaparece, aumenta a fidelidade visual e preenche os espaços vazios do cenário com micro-interações.

10. Modularização e Separação de Responsabilidades

O script atual concentra entrada de dados, física e renderização em um único arquivo. Para fins de engenharia de software, a melhoria final é a separação do código em módulos independentes. Isso facilita o teste de componentes isolados e permite que o projeto escale para múltiplas fases e tipos de inimigos sem se tornar um código impossível de manter.

Analista Responsável: Gemini AI